
TD Correction (Invariants)

Question 1 : Considérons la fonction suivante calculant la somme des n premiers entiers.

```
1 def somme_entiers(n):
2     # Pre-condition : n >= 0
3     s = 0
4     i = 0
5     while i < n:
6         i = i + 1
7         s = s + i
8     return s
```

1. Démontrer que cet algorithme termine.
2. Donner un invariant de boucle pour la boucle while.
3. Prouvez la correction partielle de cet algorithme. En conclure la correction totale.

Question 2 : Soit l'algorithme de calcul de la puissance x^n itérative via une boucle classique :

```
1 def puissance_iter(x, n):
2     # Pre-condition : n >= 0
3     res = 1
4     i = 0
5     while i < n:
6         res = res * x
7         i = i + 1
8     return res
```

1. Démontrer que cet algorithme termine.
2. Donner un invariant de boucle pour la boucle while.
3. Prouvez la correction partielle de cet algorithme. En conclure la correction totale.

Question 3 : Soit l'algorithme de division entière itérative vu lors du TD sur la terminaison :

```
1 def div_entiere(a, b):
2     # Pre-condition : a >= 0 et b > 0
3     q = 0
4     r = a
5     while r >= b:
6         r = r - b
7         q = q + 1
8     return q, r
```

On souhaite prouver qu'à l'issue de cet algorithme, le couple (q, r) forme bien la division euclidienne de a par b , c'est-à-dire $a = b \times q + r$ avec $0 \leq r < b$.

1. Démontrer que cet algorithme termine.
2. Donner un invariant de boucle pour la boucle while.
3. Prouvez la correction partielle de cet algorithme. En conclure la correction totale.

Question 4 : Considérons l'algorithme d'exponentiation rapide itérative suivant :

```
1  def expo_rapide_iter(x, n):
2      # Pre-condition : n >= 0
3      res = 1
4      a = x
5      b = n
6      while b > 0:
7          if b % 2 == 1:
8              res = res * a
9              a = a * a
10             b = b // 2
11     return res
```

1. Démontrer que cet algorithme termine.
2. Donner un invariant de boucle pour la boucle while.
3. Prouvez la correction partielle de cet algorithme. En conclure la correction totale.

Question 5 : Tri par insertion

```
1  def indice_insert(t, x):
2      for i in range(len(t)):
3          if t[i] > x:
4              return i
5      return len(t)
6
7  def tri_insertion(t):
8      res = []
9      for j in range(len(t)):
10         i = indice_insert(res, t[j])
11         # insert x dans res à l'indice i
12         res.insert(i, t[j])
13     return res
```

1. Prouver la terminaison de tri_insertion à l'aide de variants.
2. Donner un invariant et faites la correction de indice_insert.
3. Donner un invariant et faites la correction de tri_insertion.
4. En conclure la correction totale.

Question 6 : Tri bulle

```
1 def tri_bulle(t):
2     n = len(t)
3     for i in range(n):
4         for j in range(0, n - i - 1):
5             if t[j] > t[j + 1]:
6                 t[j], t[j + 1] = t[j + 1], t[j]
7     return t
```

1. Prouver sa terminaison à l'aide de variants.
2. Donner un invariant et faites la correction de la boucle interne.
3. Donner un invariant et faites la correction de la boucle externe.
4. En conclure la correction totale.